



## 石河子大学信息科学与技术学院 《智能计算系统》课程实验报告

|       |               |
|-------|---------------|
| 实验名称: | 实验二           |
| 学生姓名: | 卢瑶            |
| 学 号:  | 20211018341   |
| 学 院:  | 信息科学与技术学院     |
| 专业年级: | 计科 2021 级 5 班 |

## 实验二 手写数字识别

### 一、实验目的：

- 1、掌握 MNIST 数据集
- 2、掌握 Sequential 的编程流程
- 3、编写程序实现常用的操作

### 二、实验要求：

- 1、用 Tensorflow 编程框架和 Python 语言实现源程序的编写；
- 2、源程序应书写规范，适当添加注释；
- 3、实验课对源程序进行编辑、调试、运行；实验结束后尽快完成实验报告，并按规定时间上交实验报告。

### 三、实验内容

编写程序实现手写数字识别。

### 四、实验结果

#### 4.1 实验代码

##### (1) 数据集可视化代码

代码片段如下，其目的是加载 MNIST 数据集并可视化第一个训练样本，同时打印相关的数据维度信息。

```
1. import tensorflow as tf
2. from matplotlib import pyplot as plt
3.
4. mnist = tf.keras.datasets.mnist
5. (x_train, y_train), (x_test, y_test) = mnist.load_data()
6.
7. # 可视化训练集输入特征的第一个元素
8. plt.imshow(x_train[0], cmap='gray') # 绘制灰度图
9. plt.show()
10.
11. # 打印出训练集输入特征的第一个元素
12. print("x_train[0]:\n", x_train[0])
13. # 打印出训练集标签的第一个元素
14. print("y_train[0]:\n", y_train[0])
15.
16. # 打印出整个训练集输入特征形状
17. print("x_train.shape:\n", x_train.shape)
18. # 打印出整个训练集标签的形状
```

```

19. print("y_train.shape:\n", y_train.shape)
20. # 打印出整个测试集输入特征的形状
21. print("x_test.shape:\n", x_test.shape)
22. # 打印出整个测试集标签的形状
23. print("y_test.shape:\n", y_test.shape)

```

我们使用了 TensorFlow 和 Matplotlib 库来处理 MNIST 数据集。使用 `mnist.load_data()` 方法加载 MNIST 数据集。这个数据集包含有手写数字图片的训练集和测试集。这里将训练集和测试集分别存储到 `(x_train, y_train)` 和 `(x_test, y_test)` 两个元组中。其中，`x_train` 和 `x_test` 分别是训练集和测试集的输入特征，是  $28 \times 28$  的灰度图像像素值矩阵，灰度图像只有一个通道，输入大小则为  $28 \times 28 \times 1$ ，若为 rgb 彩色图像则有三个通道，输入即为  $28 \times 28 \times 3$ ，`y_train` 和 `y_test` 分别是训练集和测试集的标签，是 0-9 的数字标签。

接着，打印出训练集输入特征的第一个元素，即 `x_train` 的第一张图的像素矩阵。实验中是一个  $28 \times 28$  的灰度图像像素矩阵，所以共有 784 个输入特征。实验中的输入标签为 5。

训练集中包含 60000 个  $28 \times 28$  像素的图像，测试集中包含 10000 个  $28 \times 28$  像素的图像。

## (2) 多层感知机训练代码

我们创建并训练了一个简单的多层感知机 (MLP)，用于 MNIST 数据集上的手写数字分类。

```

1. import tensorflow as tf
2.
3. mnist = tf.keras.datasets.mnist
4. (x_train, y_train), (x_test, y_test) = mnist.load_data()
5. x_train, x_test = x_train / 255.0, x_test / 255.0
6.
7. model = tf.keras.models.Sequential([
8.     tf.keras.layers.Flatten(),
9.     tf.keras.layers.Dense(128, activation='relu'),
10.    tf.keras.layers.Dense(10, activation='softmax')
11. ])
12.
13. model.compile(optimizer='adam',
14.               loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),

```

```
15.                                     metrics=['sparse_categorical_accu
    racy'])
16.
17.     model.fit(x_train, y_train, batch_size=32, epochs=5, vali
    dation_data=(x_test, y_test), validation_freq=1)
18.     model.summary()
19.
```

我们首先把 MNIST 数据集划分为训练集和测试集，其中 (x\_train, y\_train) 是训练集输入特征和标签，(x\_test, y\_test) 是测试集输入特征和标签。

再对训练集和测试集进行预处理，将像素值从 0-255 缩放到 0-1 之间，缩放可以将输入数据的范围标准化到 0-1 之间，这有助于优化算法的收敛速度和效率。

然后构建一个序列模型，其中包含一个 Flatten 层、一个具有 128 个神经元的全连接层和一个具有 10 个神经元的全连接层。Flatten 层：该层用于将输入的二维数组数据展开成一维数组，以便进行全连接的操作。Dense 层（隐藏层）：该层是一个全连接层，有 128 个神经元。使用 ReLU 激活函数来引入非线性性。ReLU 是一种常用的激活函数，其形式为  $f(x) = \max(0, x)$ 。Dense 层（输出层）：该层也是一个全连接层，有 10 个神经元，分别对应 0-9 的数字分类。使用 softmax 激活函数来将输出转化为每个类别的概率，以便进行多分类的操作。

然后配置模型的训练参数，包括优化器、损失函数和评估指标。

优化器：优化器是用来更新神经网络模型参数的算法，目的是最小化损失函数。这里我们使用 Adam 优化器，它是一种自适应学习率算法，能够在不同层之间自适应地调整学习率，以便更快地收敛到全局最优解。

损失函数：损失函数用来衡量模型的预测值与真实值之间的差异，目的是最小化误差。在这里我们使用 SparseCategoricalCrossentropy 作为损失函数，它适用于多分类问题，并且可以自动将类别标签转化为 one-hot 编码。当 from\_logits 参数为 False 时，SparseCategoricalCrossentropy 会自动将输出层的 softmax 结果转化为概率分布，用于计算损失值。

评估指标：评估指标用于衡量模型的预测准确率。在这里我们使用 sparse\_categorical\_accuracy，它是一种稀疏分类准确率指标，用于多分类问题。它会将类别标签转化为整数编码，然后计算预测值与真实值之间的匹配率。

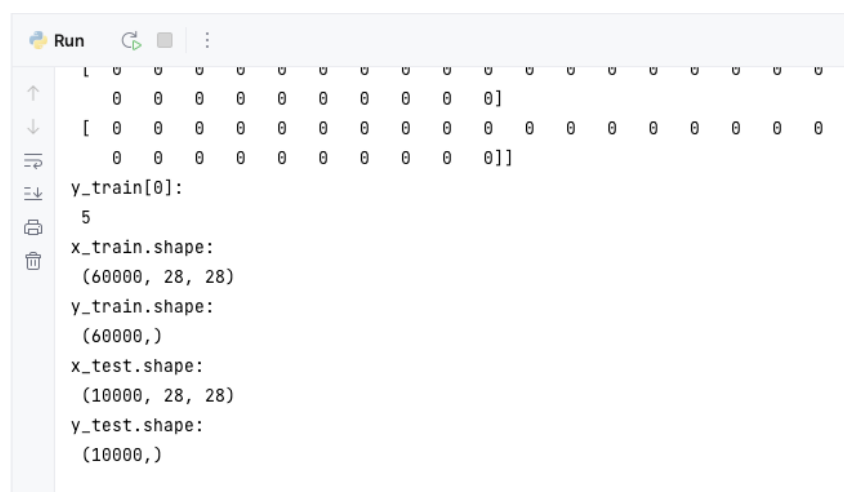
再进行模型的训练，使用训练集训练模型，其中 batch\_size 指定批次大小，

epochs 指定训练的轮数，validation\_data 指定验证集，validation\_freq 指定在训练过程中进行验证的频率。model.fit() 会对模型进行训练，并返回训练过程中的各项指标。在每个 epoch 完成后，训练过程中的损失函数和指定的评估指标会打印出来。同时，如果指定了验证集，每个 epoch 结束后，模型会对验证集进行评估，并将评估结果打印出来。

最后打印模型的结构和参数信息，包括每层的名称、每层神经网络节点数和参数数量。

## 4.2 实验运行结果

### (1) 数据集可视化结果



```
Run [ 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
0 0 0 0 0 0 0 0 0 0 0]]
y_train[0]:
5
x_train.shape:
(60000, 28, 28)
y_train.shape:
(60000,)
x_test.shape:
(10000, 28, 28)
y_test.shape:
(10000,)
```

图 1 数据集量化结果

从以上运行结果可以看到：

1、训练集和测试集的大小：训练集中有 60000 个样本，测试集中有 10000 个样本。

2、图像大小：每个图像的大小为  $28 \times 28$  像素，共 784 个像素。

数据集集中的第一张图片如下所示：

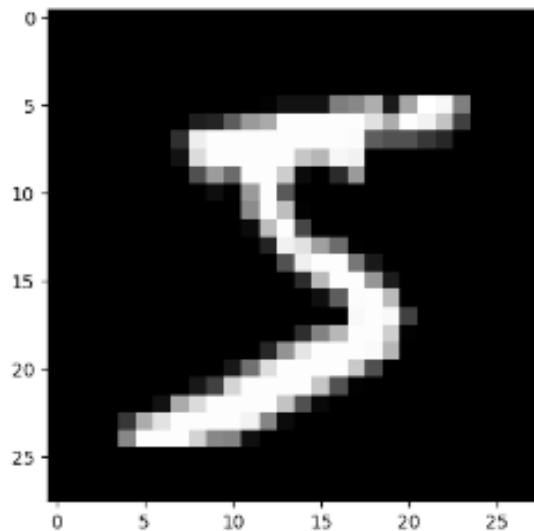


图 2 MINIST 数据集第一张图片

## (2) 多层神经网络训练结果图

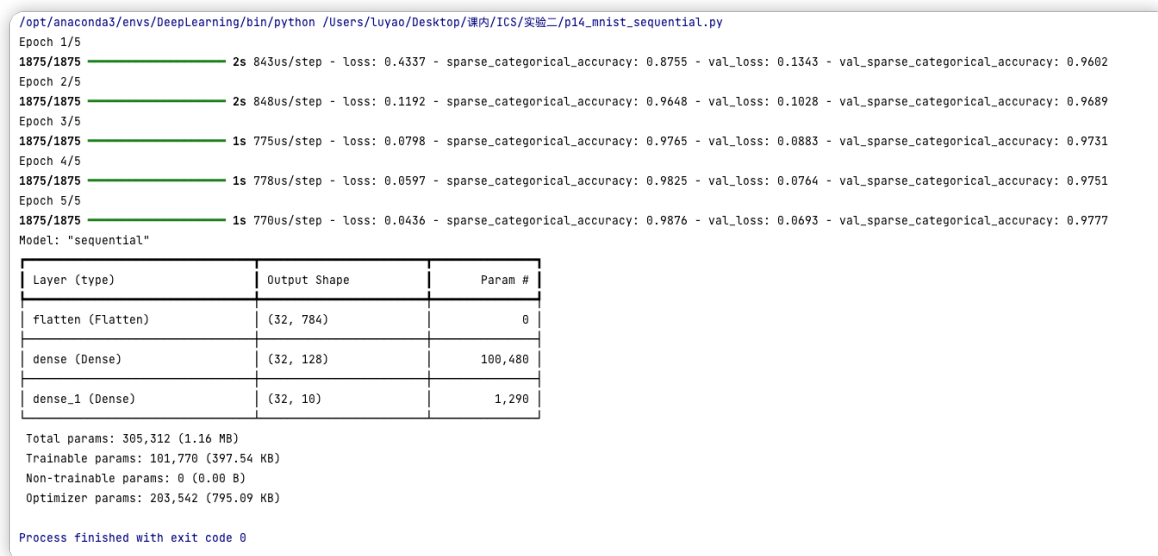


图 3 多层神经网络训练结果

对结果分析我们得到：

- **Epochs:** 模型已经通过了五个训练周期 (epoch)，每个周期都处理了整个训练集。Epoch 计数从 1 开始。
- **训练进度:** 每个 epoch 的训练进度以及该周期中的损失和稀疏分类准确率 (sparse\_categorical\_accuracy) 均有显示。损失是模型在训练数据上的性能评估，而准确率是模型正确分类图像的比例。
- **损失和准确率:** 随着 epoch 的增加，可以看到损失在逐渐减小，而稀疏分

类准确率在提高，这表明模型正在学习并且其性能在提升。

- **验证频率:** 由于设置了 `validation_freq=1`，模型在每个 epoch 结束时都会在测试集上进行验证，并报告验证损失 (`val_loss`) 和验证稀疏分类准确率 (`val_sparse_categorical_accuracy`)。

- **模型概览:** 最后，模型的结构总结表明模型由一个 Flatten 层和两个 Dense 层组成。Flatten 层负责将 28x28 的图像数据压平成一维数组，而 Dense 层则作为神经网络的主要组成部分。第一个 Dense 层有 128 个神经元和激活函数 ReLU，第二个 Dense 层是输出层，有 10 个神经元，对应于 10 个数字类别，并使用 softmax 激活函数。

- **参数数量:** 模型总共有 101,770 个可训练参数，这是模型在训练过程中需要学习的权重和偏置的总数。

## 五、实验小结（重点部分，从数据集、流程图、神经网络模型等方面进行小结）

### 5.1 数据集

MNIST 数据集是机器学习中广泛使用的一个基础数据集，用于手写数字的分类任务。它包含了 60,000 个训练样本和 10,000 个测试样本，每个样本都是一个 28x28 像素的灰度图像，对应于数字 0 到 9。每个图像都附有一个标签，指示了图像中手写数字的真实类别。在本次实验中，这些图像通过多层神经网络进行处理，网络通过学习图像与标签之间的映射来预测新样本的数字。

## 5.2 流程图

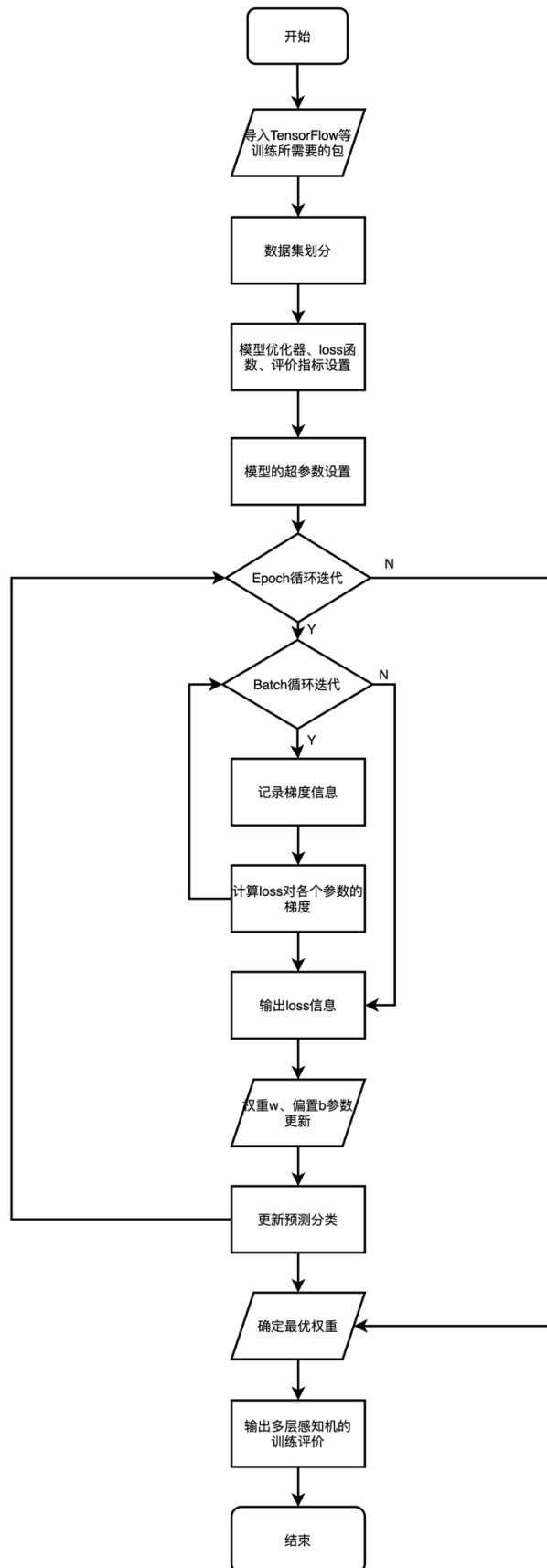


图 4 多层感知机训练流程图



流程图的文字描述如下所示：

第一步，导入 TensorFlow 库

第二步，加载 MNIST 数据集，拆分成训练集和测试集，同时对像素值进行归一化

第三步，定义 Sequential 模型

第四步，添加 Flatten 层，将输入图像展平成一维向量

第五步，添加 Dense 层，该层包含 128 个神经元，并使用 ReLU 激活函数

第六步，添加 Dense 层，该层包含 10 个神经元，并使用 Softmax 激活函数

第七步，编译模型，指定损失函数为 SparseCategoricalCrossentropy，优化器为 Adam，指定使用精确度作为评估指标

第八步，使用 `fit()` 函数训练模型，指定训练数据、批次大小、迭代次数和验证数据

第九步，模型在训练数据上迭代 5 次，同时在验证数据上进行验证

第十步，输出模型的总体信息、网络架构和每一层的输出形状、参数数目等信息

### 5.3 多层感知机

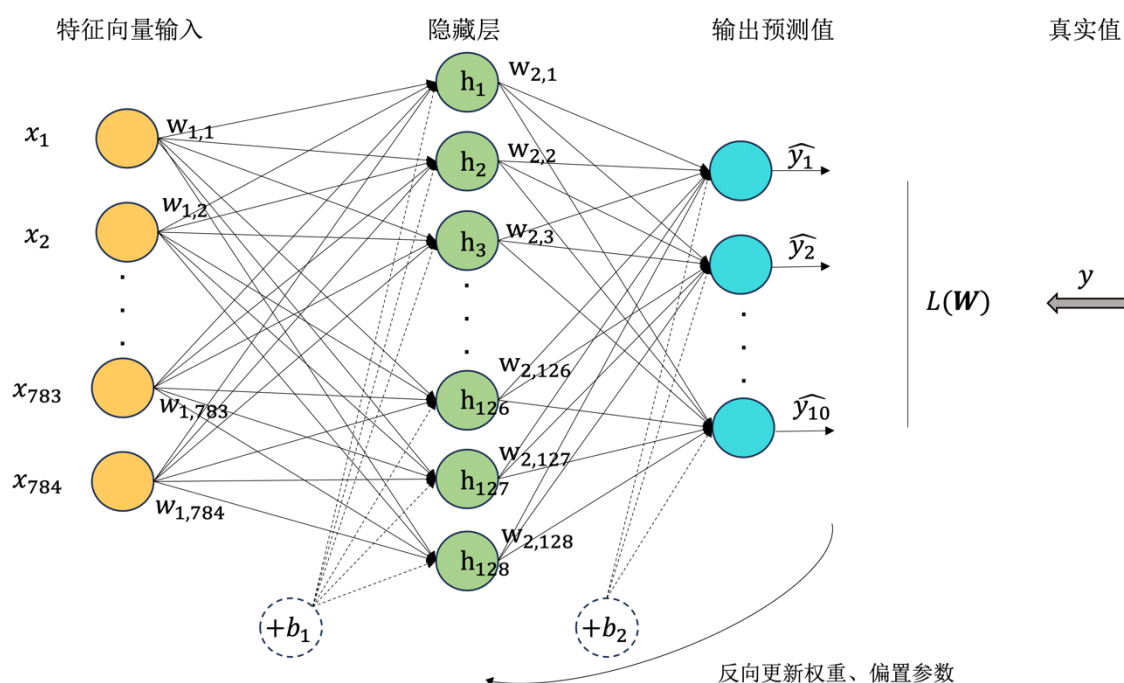


图 5 多层感知机网络结构图

从上图我们可以看到：

### 1. 输入层 (Input Layer):

- 输入层由图像像素组成，每个像素对应一个输入节点。在这个图像中，输入层包含了  $28 \times 28 = 784$  个节点，表示一个  $28 \times 28$  像素的手写数字图像。

### 2. 隐藏层 (Hidden Layers):

- 图中显示了两个隐藏层，每个隐藏层包含多个节点。
- 每个节点通过连接权重与前一层的所有节点相连，并通过激活函数进行非线性转换。

### 3. 输出层 (Output Layer):

- 输出层包含了 10 个节点，每个节点表示一个数字类别(0 到 9)的概率。
- 输出层的激活函数通常是 softmax 函数，用于将输出转换为概率分布。

### 4. 连接权重 (Connection Weights):

- 每个节点之间的连接都有一个权重，这些权重决定了信号在网络中传播时的重要性。
- 权重在训练过程中通过反向传播算法进行调整，以最小化损失函数并提高模型的性能。

### 5. 损失函数 (Loss Function):

- 损失函数用于衡量模型的预测与实际标签之间的差异。
- 在训练期间，模型试图最小化损失函数，以使预测结果尽可能接近真实标签。

### 6. 优化器 (Optimizer):

- 优化器是用于调整连接权重的算法，以最小化损失函数。
- 常见的优化算法包括随机梯度下降 (SGD) 和 Adam 优化器。

## 六、成绩